Uncovering the Intricacies and Synergies of Processor Microarchitecture Mechanisms Using Explainable AI

Abdoulaye Gamatié, Yuyang Wang, and Diego Valdez Duran

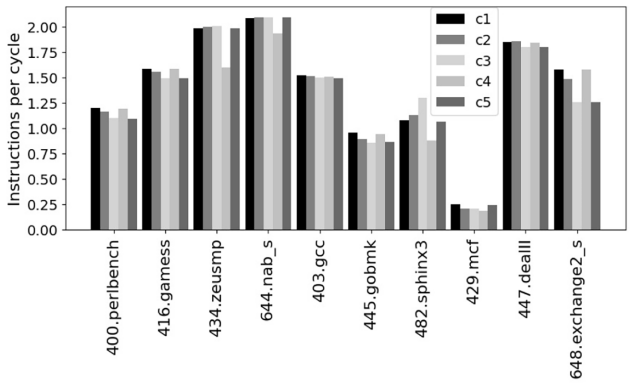
***Abstract*—This paper defines a data-driven methodology seamlessly combining machine learning (ML) and eXplainable Artificial Intelligence (XAI) techniques to address the challenge of understanding the intricate relationships between microarchitecture mechanisms with respect to system performance. By applying the SHapley Additive exPlanations (SHAP) XAI method, it analyzes the synergies of cache replacement, branch prediction, and hardware prefetching on instructions per cycle (IPC) scores. We validate our methodology by using the SPEC CPU 2006 and 2017 benchmark suites with the ChampSim simulator. We illustrate the benefits of the proposed methodology and discuss the major insights and limitations obtained from this study.**

***Index Terms*—Machine learning, processor microarchitecture, explainable artificial intelligence, performance improvement.**

I. Introduction

In the ever-evolving landscape of processor microarchitecture, optimizing performance remains a central goal. Achieving this objective requires the careful selection and configuration of hardware mechanisms. These mechanisms often interact indirectly to improve system performance. Their effects generally depend on workload nature. Different programs can vary widely in their memory access patterns, instruction flow, and computational demands. So, a specific hardware configuration that fits one program may prove less adequate for another.

Fig. 1 shows the impact of various processor microarchitecture configurations on system performance, specifically *instructions per cycle* (IPC), based on SPEC CPU 2006 and 2017 benchmarks. We use the ChampSim simulator [1] to evaluate each benchmark on a single-core processor (see Table II for setup details) across five configurations, focusing on branch prediction, prefetching in L1-I, L1-D, L2 and L3 caches, and



	Branch pred.	L1-D prefetch	L1-I prefetch	L2 prefetch	L3 prefetch	L3 replac.
c1	*hashed perc.*	*no*	*next line*	*next line*	*no*	*DRRIP*
c2	*perceptron*	*next line*	*no*	*next line*	*no*	*LRU*
c3	*bimodal*	*next line*	*no*	*no*	*next line*	*Hawkeye*
c4	*hashed perc.*	*no*	*no*	*no*	*no*	*SHiP*
c5	*bimodal*	*next line*	*next line*	*ip-stride*	*no*	*DRRIP*

Fig. 1. IPC variation for 10 SPEC CPU benchmarks w.r.t. branch predictors, prefetchers, and cache replacement at LLC. Each configuration c_k is defined according to six microarchitecture mechanisms in the above table.

L3 cache replacement. Branch prediction involves forecasting whether a branch will be taken before it is executed. It therefore mitigates branching costs in CPUs using pipelining. Prefetching brings data and instructions to the CPU before request. Cache replacement determines which cache memory blocks are replaced with new data requested by the CPU after a cache miss. To explore the impact of processor configurations[1] on IPC, we considered the following branch prediction techniques: *bimodal* [2], *gshare* [3], *perceptron* [4], and *hashed perceptron* [5]. The evaluated prefetch techniques are *next line* and *stride-based*. The former prefetches the next cache line on a cache miss or prefetch hit. The latter determines a stride in an accessed address to make a prefetch request. When no prefetcher is applied, this is denoted by "*no*" in Fig. 1. Regarding cache replacement policies, we evaluated *least recently used* (LRU), *dynamic re-reference interval prediction* (DRRIP) [6], *static re-reference interval prediction* (SRRIP) [6], *signature-based hit predictor* (SHiP) [7] and *Hawkeye* [8]. Each evaluated configuration is denoted by these mechanisms.

[1]For the sake of simplicity, we deliberately use a subset of existing branch prediction, prefetching, and cache replacement techniques. Alternatively, other techniques can be considered to illustrate similar results.

Received 14 January 2024; revised 10 September 2024; accepted 29 October 2024. Date of publication 18 November 2024; date of current version 20 January 2025. Recommended for acceptance by T. E. Carlson. *(Corresponding author: Abdoulaye Gamatié.)*

Abdoulaye Gamatié and Yuyang Wang are with LIRMM - University of Montpellier and CNRS, 34095 Montpellier, France (e-mail: abdoulaye.gamatie@lirmm.fr).

Diego Valdez Duran was with LIRMM - University of Montpellier and CNRS, 34095 Montpellier, France, and also with the University of Stanford, Palo Alto, CA 94305 USA.

Digital Object Identifier 10.1109/TC.2024.3500377

Fig. 1 reports IPC scores that fluctuate non-monotonically across various benchmarks and hardware configurations. Scenarios with advanced branch prediction, prefetching, and cache replacement techniques, e.g., hashed perceptron, stride-based, and Hawkeye, can be outperformed by more basic techniques depending on the combined hardware mechanisms. This emphasizes the unique complexity of devising a one-size-fits-all design that maximizes performance across various workloads.

Addressed challenge. We are interested in uncovering some intricate relationships between processor microarchitecture mechanisms for system performance. Performance is fundamentally determined by how efficiently instructions can be executed and how memory accesses can be cached. Prefetching, branch prediction, and cache replacement mechanisms play a synergistic role in performance optimization. Addressing their interaction is a significant challenge that deserves all our attention [9], [10]. A holistic understanding of the involved complexities requires powerful design exploration approaches. This work will focus on the following research questions:

- (RQ1) *By accounting for multiple microarchitecture mechanisms at once, including their complex and non-linear interactions, how can we holistically formulate the aforementioned problem?*
- (RQ2) *How can we leverage the resulting formulation to analyze both the isolated and synergistic effects of these mechanisms on system performance?*
- (RQ3) *Which insights and limitations could we draw from our answers to the above questions?*

Rationale and adopted vision. Modern processor microarchitectures are complex systems that can be addressed with design methods inspired by *general system theory* [11]. *Analytical* (or Cartesian) methods break the system into individual components. They focus on specific interactions. By modifying one element at a time, they provide detailed insights and help identify issues and areas of improvement. However, this narrow focus can sometimes miss the broader context of the system. This can lead to incomplete solutions. In contrast, *holistic* methods consider the system as an integrated whole. They examine all interactions and their broader implications. This requires changing multiple elements at once to understand their interdependencies. Although more challenging, holistic methods offer a comprehensive view. This makes them ideal for tackling the aforementioned challenge.

This paper promotes a holistic approach by adopting a *data-driven* reasoning through *machine learning* (ML) techniques given their promises in computer architecture design and optimization [12], [13], [14], [15], [16], [17]. It also relies on *eXplainable Artificial Intelligence* (XAI) [18], which enables one to uncover the underlying knowledge captured by ML models. The state-of-the-art on ML-based microarchitecture design only considers accurate prediction (see Section II). ML model explanations w.r.t. decision-making are out of their scope, hindering actionable insights. Since the present work mainly focuses on methodological issues, we deliberately adopt a pragmatic simulator to model and evaluate the target designs. We use ChampSim [1], which is less accurate than the full-system gem5 simulator [19] or real hardware platforms, yet faster and relevant to illustrate our proposed methodology.

Our contribution. This paper proposes a design assistance solution for computer architects summarized as follows:

- (Answer to RQ1) We devise a methodology that seamlessly combines ML and XAI to address the challenge of understanding the intricate relationships between processor microarchitecture mechanisms w.r.t. system performance. It consists of three steps: 1) model and evaluate the IPC of different processor configurations in a comprehensive way; 2) train relevant neural networks (NNs) from the evaluation results for IPC score prediction[2]; and 3) apply a state-of-the-art XAI method to analyze the combined effects of the mechanisms on IPC prediction.
- (Answer to RQ2) We validate our methodology on the SPEC CPU 2006 and 2017 benchmark suites [20], [21] with ChampSim [1]. By applying the *SHapley Additive exPlanations* (SHAP) method [22], we study the impact of cache replacement, branch prediction, and hardware prefetching, as well as their interplay, w.r.t. IPC prediction. This allows us to analyze the synergistic effects of these mechanisms on performance across benchmarks.
- (Answer to RQ3) We discuss the major insights and limitations of our methodology. We shed light on the importance of balancing accurate performance predictions with understanding and addressing the limitations of ML models in robust system design.

The above contribution opens a promising opportunity for computer architects to address highly complex questions that are not yet well covered by the literature. We believe this will pave the way for an unprecedented quantitative approach to (micro)architecture design in era of modern AI.

Outline. We first discuss some related work in Section II. Then, we present our methodology in Section III. We illustrate this methodology in a case study in Section IV, by showing how to build NNs and analyze their feature contribution to IPC prediction with SHAP. In Section V, we discuss some gained insights and limitations of our approach. Finally, we give concluding remarks and perspectives in Section VI.

II. Related Work

Prior work on microarchitecture design concentrated on performance enhancement by refining fundamental hardware mechanisms. A major part addresses one mechanism at a time [4], [23], [24], [25], [26], [27], [28], [29], while a smaller portion focuses on the interplay between multiple mechanisms [9], [10], [30], [31]. ML has been primarily applied to optimize cache replacement, prefetching, and branch prediction. It has recently been used to simultaneously deal with several mechanisms [32] or support design space exploration [12].

A. Addressing Single Mechanism With ML Techniques

Shi et al. demonstrated the benefit of LSTM model-based prediction in the so-called Glider cache replacement policy [23].

[2]In comparison with a fast simulation with ChampSim, a trained neural network takes significantly less time to infer IPC scores.

They showed Glider outperforms state-of-the-art policies in terms of IPC improvement, e.g., 14.7% (LRU), 13.6% (Hawkeye), 11.4% (SHiP++). Teran et al. concentrated on block reuse prediction for cache replacement, employing perceptron learning in their study [24]. Perceptron learning allows to identify independent correlations among multiple features related to block reuse. Their solution achieved a speed-up of 6.1% on SPEC CPU 2006 compared to LRU and outperforms SHiP by 3.8%. Similarly, Sethumurugan et al. leveraged reinforcement learning to develop a cost-effective cache replacement policy named Reinforcement Learned Replacement [25], improving single-core and four-core system performance by 3.25% and 4.86%, respectively, compared with LRU.

Peled et al. introduced the concept of Semantic Locality to characterize memory access patterns independently of data layout [26]. They defined a context-based memory prefetcher that approximates semantic locality through reinforcement learning (RL). This prefetcher outperforms leading spatio-temporal prefetchers. In another study, Bhatia et al. explored Perceptron-based Prefetch Filtering (PPF) to enhance prefetch coverage without sacrificing accuracy [27]. PPF enhances performance on memory-intensive benchmarks by 3.78% for a single-core and by a notable 11.4% for a 4-core system when compared to using only the underlying prefetcher.

Using a perceptron instead of conventional two-bit counters, Jimenez et al. proposed an innovative approach to branch prediction [4]. Their method achieves increased accuracy by harnessing extensive branch histories while maintaining hardware resources that scale linearly. They reported a reduction of up to 10.1% in misprediction rates for SPEC CPU 2000. In [28], Zouzias et al. discussed the formulation of different branch predictors based on RL. They argued that RL could provide a way of systematically developing novel branch predictors. Finally, Zangeneh et al. exploited convolutional neural networks (CNNs) to design the BranchNet predictor [29]. They showed that BranchNet helps address branches that are difficult to predict with TAGE branch predictor [33].

B. Addressing the Synergy of Multiple Mechanisms

Jimenez et al. studied cache replacement and prefetching techniques at the Last-Level Cache [9]. The choice of a prefetcher plays a pivotal role in shaping replacement policy effectiveness. The authors introduced an innovative and cost-effective replacement policy based on insertion and promotion vectors (IPVs). Through genetically evolved demand, insertion of prefetchers, and promotion vectors, they demonstrated synergy between prefetchers and replacement policies. This led to better speed-up than state-of-the-art replacement policies. Kim et al. introduced a cache management technique coined Kill-the-PC (KPC), aiming to enhance performance by reconstructing program behavior within the cache hierarchy [10]. Their approach achieved an increase of 9.2% in performance compared with a baseline system with an advanced prefetcher.

Backes et al. proposed new inclusive and exclusive cache policies and evaluated different prefetchers, replacement policies, and core counts for each policy [30]. They observed that cache management techniques that perform well under one inclusion policy might not work under another. This highlights how challenging it is to identify optimal solutions. Yadav et al. combined multiple prefetchers to better predict the memory accesses of a CPU [31]. Bloom filters are used to track suitable prefetchers through associated scores. The authors claim a 44.29% performance improvement for single-core with SPEC CPU 2017. Yang et al. introduced a prefetch-adaptive intelligent cache (PAIC) replacement policy [32]. They utilized support vector machines (SVM) to optimize the prediction of cache block reuse in the context of prefetching. They showed that PAIC outperforms advanced policies like SHiP and Hawkeye.

Unlike the above studies, Krishnan et al. devised the ArchGym framework implementing ML-assisted architecture design space exploration [12]. This framework enables a designer to easily tune and evaluate various ML algorithms for building high-fidelity cost models (latency, power and energy), which are faster than cycle-accurate simulators.

The aforementioned works are summarized in Table I, according to target hardware mechanisms, validation support, used benchmarks, and ML techniques whenever relevant. The present work distinguishes itself from them by combining ML with XAI. It offers valuable insights into the interplay between critical microarchitecture mechanisms.

C. Towards XAI-Oriented Vision

XAI is applied in various domains, e.g., decision support, predictive maintenance, and anomaly detection [18]. No consensus exists yet on the meaning of *explainability versus interpretability* [34]. The definition suggested in [34], however, seems reasonable: "*explainability provides insights to a targeted audience to fulfill a need, whereas interpretability is the degree to which the provided insights can make sense for the targeted audience's domain knowledge.*"

Explanations can be generated at different ML stages. *Ante hoc* XAI methods produce explanations early in the training process and are well-suited to transparent ML models, such as k-nearest neighbors, linear regression, and decision trees. Conversely, *post hoc* XAI methods generate explanations during inference using interpretable surrogate models, often applied to complex models like deep neural networks. These methods can be either *model-agnostic*, compatible with any ML model, or *model-specific*. XAI explanations can be *local* or *global*. Local explanations clarify why a model makes a specific prediction for a particular data instance [35]. Global explanations aim to elucidate how the model generates predictions in a broader sense, drawing insights from its entire training dataset.

To our knowledge, only two existing works deal with XAI and computer architecture. In [36], the focus is on accelerating XAI processes using hardware accelerators, particularly to address transparency concerns with ML models. In another work [37], the concept of explainable hardware is introduced to achieve system-level explainability through XAI, though it remains in its early stages. Our study deals with a different problem and exploits XAI to support a deep analysis of processor microarchitecture's intricate dynamics. We rely on the SHAP

TABLE I
SUMMARY OF SELECTED RELATED WORK ON MICROARCHITECTURE LEVEL DESIGN FOR COMPUTER PERFORMANCE

	Target Hardware Mechanisms	Validation Support	Validation Benchmarks	ML Techniques
Shi et al. [23]	Cache replacement	ChampSim	SPEC 2006, SPEC 2017, and GAP	RNN
Teran et al. [24]	Cache replacement	In-house simulator	SPEC 2006	Perceptron
Sethumurugan et al. [25]	Cache replacement	ChampSim and Python	SPEC 2006 and CloudSuite	Reinforc. learning
Jimenez et al. [4]	Branch prediction	AMD K6-III simulator	SPEC 2000	Perceptron
Zouzias et al. [28]	Branch prediction	CPB-5 simulator	CBP-5 associated traces	Reinforc. learning
Zangeneh et al. [29]	Branch prediction	Scarab cycle-level simulator for x86-64	SPEC 2017	CNNs
Peled et al. [26]	Cache prefetching	gem5 simulator	SPEC2006, Graph500, PBBS and HPCS suites	N/A
Bhatia et al. [27]	Cache prefetching	ChampSim	SPEC 2017	Perceptron
Jimenez et al. [9]	Cache replacement, prefetching	ChampSim	SPEC 2006, SPEC 2017, GAP and XSBench	Genetic algorithm
Kim et al. [10]	Cache replacement, prefetching	ChampSim	SPEC 2006, CloudSuite and ML workload mlpack_cf	N/A
Backes et al. [30]	Cache replacement, cache inclusion/exclusion, prefetching	ChampSim	SPECspeed 2017, CloudSuite and ML workload mlpack_cf	N/A
Yadav et al. [31]	Different prefetching	Trace-based simulation	SPEC CPU 2017	N/A
Yang et al. [32]	Cache replacement, prefetching	ChampSim	SPEC C2006, SPEC 2017	SVM
Our work	Cache replacement, prefetching, branch prediction	ChampSim	SPEC 2006 and SPEC 2017	NNs and XAI

method [22], which supports both global and local model explanations. It provides a principled way to interpret ML models' output by assigning each feature's contribution to the model's prediction. SHAP values are based on cooperative game theory and aim to fairly distribute the *credit* for the prediction among the input features. Formally speaking, let us consider a reference set Φ of features. SHAP computes the *Shapley value* of a feature $\phi_i \in \Phi$, w.r.t. a *feature vector* $\phi = \langle \phi_1, \ldots, \phi_k \rangle$ and a prediction model f. This value is the average contribution of feature ϕ_i to predictions made by f w.r.t. ϕ. It is defined by function Ψ_i [22]:

$$\Psi_i(f, \phi) = \Sigma_{z' \subseteq \phi'} \frac{|z'|!(|\Phi| - |z'| - 1)!}{|\Phi|!} [f_\phi(z') - f_\phi(z' \backslash \phi_i)] \tag{1}$$

where ϕ' denotes a simplified binary input feature vector that maps to an original input feature vector ϕ through the specific function m_ϕ. The variable z' belongs to the set $\{0, 1\}^{|\Phi|}$ where $|\Phi|$ is the number of features in Φ. The notation $|z'|$ expresses the number of 1's in the binary vector z', and $z' \subseteq \phi'$ represents the set of all vectors z' where the 1's are a subset of those present in ϕ'. Here, $f_\phi(z')$ is equivalent to $f(m_\phi(z'))$. The expression $[f_\phi(z') - f_\phi(z' \backslash \phi_i)]$ defines the marginal contribution of feature ϕ_i to the prediction model f w.r.t. a vector z'. Note that $z' \backslash \phi_i$ means the value at position i in z' is set to 0. When evaluating f in feature vectors with fewer elements than the original input vectors, missing features are commonly assigned default values.

In essence, SHAP breaks down the black-box nature of complex ML models and offers transparency by quantifying the impact of each input variable.

Beyond XAI, visualization methods can help explain hardware by converting complex data into intuitive and interactive visual formats. A study placed computer architects at the forefront of a design process, supported by visualization experts, for analyzing Network-on-Chip behavior in simulated chip designs [38]. Its broader applicability may be limited by the need for active domain expert involvement. Another study introduced a tool for visualizing GPU performance based on execution traces from simulators [39]. It allows one to identify performance bottlenecks and explore system improvements. The tool's complexity and focus on GPUs could be a barrier to users interested in other hardware analysis contexts.

III. METHODOLOGY

We propose a methodology that involves three main steps: (1) modeling the target designs and collecting evaluation data, (2) defining ML models that accurately predict performance scores based on certain architectures and benchmarks, and (3) using XAI to explain ML models' predictions. Each step is summarized below (see details in Section IV).

Step 1: Design modeling and training data generation. Using ChampSim [1], a trace-based simulation tool designed for investigating memory hierarchy, we model and evaluate a wide range of processor microarchitectures. ChampSim simulation utilizes program execution traces as input to simulate program executions on user-defined microarchitectures. It produces useful performance metrics like IPC and cache hits/misses. Various parameters can be adjusted to explore how they affect system performance. These include cache size, associativity, and access latency. Here, we focus on branch prediction, cache hardware prefetching, and replacement policy by evaluating some corresponding instances. We report the resulting processor performance scores.

Step 2: IPC prediction modeling based on neural networks. NNs are relevant for intricate, high-dimensional tasks like predicting IPC scores from microarchitecture data. To ensure accurate prediction, the training process must overcome *overfitting*. An overfitted model typically performs well on training data but poorly on unseen data. We implement a dedicated NN for each benchmark program together with its associated subset of microarchitecture configurations.

The reason behind this choice is that several benchmarks behave differently on the same hardware configurations. Defining a single NN model to predict IPC for all benchmarks can hide the distinct behavioral patterns and characteristics inherent to each benchmark. Given the manageable size of the SPEC CPU benchmark suites, we opted to train separate NNs for each benchmark in this study. This approach captures the distinct characteristics of each benchmark, leading to more accurate IPC predictions. However, these benchmark-specific NNs have limited generalizability to unseen workloads. To overcome this, a single NN could be trained using relevant workload characteristics, which can be derived from profiling or extrapolated from carefully selected workloads. By associating performance metrics with these characteristics instead of specific benchmarks, the model can more easily handle new, unseen workloads that share similar traits. Although profiling SPEC CPU benchmarks is complex and beyond the scope of this study, this alternative NN training strategy could be explored in future work.

Understanding the outcomes of defined NNs requires adequate analysis from both ML and processor microarchitecture perspectives. This analysis is itself sensitive to the ChampSim simulator's accuracy. Typically, we must be able to distinguish behaviors inherent to ChampSim data from the approximated behaviors learned by NNs from these data. All these considerations make XAI sometimes produce difficult-to-interpret NN explanations. To address this issue, we build an *Oracle* system that accurately knows the IPC score for each workload and microarchitecture configuration. Creating such an Oracle is challenging for actual processors due to hardware complexity, but here, we leverage ChampSim outputs. The Oracle serves as a sanity check for NN consistency, with minimal discrepancies being the desirable objective.

Step 3: Model explanations using XAI. This part analyzes the impact of microarchitecture mechanisms on IPC prediction. We leverage SHAP analysis to compare NN behavior explanations against Oracle ones. A high degree of fidelity means that the NN models are consistent enough w.r.t. Oracle. Afterward, we rely on consistent NNs to analyze the contribution of the microarchitecture mechanisms as well as their interplay w.r.t. IPC prediction.

IV. Case Study

A. System Modeling and Training Data Generation

We propose a comprehensive experimental framework to investigate various configurations of a typical processor [40]. The main characteristics of this processor are summarized in Table II and modeled with ChampSim. The evaluated microarchitecture mechanisms include cache prefetchers, cache replacement policies, and branch predictors (see Table III). This results in $4 \times 2 \times 2 \times 3 \times 2 \times 5 = 480$ configurations, each representing a specific architecture setup to evaluate.

To assess IPC scores, we employ execution traces [41] obtained from 49 programs in the SPEC CPU 2006 [20] and SPEC CPU 2017 [21] benchmark suites. When multiple trace versions exist for a program benchmark, we select the version with the largest input size. These traces have been generated for the third data prefetching championship (DPC2) in 2019. Overall, this leads to $49 \times 480 = 23520$ simulation instances in the training dataset. On average, a simulation instance takes about 1 to 2 hours and involves 50 million warm-up instructions and 200 million simulation instructions.

TABLE II
Main Processor Parameters Modeled in ChampSim

CPU	Out-of-order, single-core, 4 GHz, LQ size = 128 entries, LQ width = 2, SQ size = 72 entries, SQ width = 2, IQ size = 64 entries, IQ width = 6, ROB size = 352 entries, number of FUs = 6 pipelined, pipeline width = 6
L1-I	32KB, 8-way, 4-cycle latency, blocksize 64, LRU
L1-D	48KB, 12-way, 5-cycle latency, blocksize 64, LRU
L2	512KB, 10-way, 10-cycle latency, blocksize 64, LRU
L3	2MB, 16-way, 20-cycle latency, blocksize 64
MSHRs	8/16/32/64 at L1-I/L1-D/L2/L3
DRAM cont.	One channel, 6400MTPS, 64-entry RQ and WQ
DRAM	t_RP: 12.5ns, t_RCD: 12.5ns, t_CAS: 12.5ns

TABLE III
Studied Microarchitecture Mechanisms and Their Algorithms

Mechanisms	Evaluated Configurations
Branch prediction (BP)	[bimodal, gshare, perceptron, hashed percep.]
L1-I prefetcher (L1IP)	[no, next line]
L1-D prefetcher (L1DP)	[no, next line]
L2 prefetcher (L2P)	[no, next line, ip stride]
L3 prefetcher (L3P)	[no, next line]
L3 cache replac. (L3C)	[LRU, SRRIP, DRRIP, SHiP, Hawkeye]

Each row of the resulting dataset records a program benchmark identifier, a vector specifying the configuration of the considered microarchitecture mechanisms (see Table III), and the corresponding IPC value. The one-hot encoded vectors of these rows are used as inputs to NNs to predict IPC.

The flow of generating the training dataset is completely automated. This dataset initially contains 8 columns (or features): the 6 microarchitecture mechanisms summarized in Table III, the benchmark names, and computed IPC scores.

Data preprocessing. After cleaning the dataset by removing incomplete rows and checking for feature correlations, we proceed with data transformation. ML algorithms typically require numerical data, so we convert categorical features, such as microarchitecture mechanisms, into numerical form. To avoid misinterpretation from ordinal assumptions, we use one-hot encoding (see Table IV). This approach creates separate binary features for each categorical option, preserving feature independence and avoiding ordinal relationships. For example, the branch predictor with four categorical labels, *bimodal, hashed perceptron, gshare,* and *perceptron*, is transformed into a vector of four binary values, where each binary feature indicates the presence of a specific category.

One-hot encoding can increase dataset dimensionality, potentially complicating model training due to sparse input vectors dominated by zeros. However, in this study, the three categorical features only expand to 12 binary features out of 16 total, minimizing this issue.

TABLE IV
ONE-HOT ENCODING OF THE DIFFERENT POLICIES

Branch Prediction	Bimodal	Hashed perc.	gshare	Perceptron
BP = bimodal	1	0	0	0
BP = hashed perc.	0	1	0	0
BP = gshare	0	0	1	0
BP = perceptron	0	0	0	1

L2 cache prefetching	No	IP stride	Next line
L2P = no prefetch	1	0	0
L2P = IP stride	0	1	0
L2P = next line	0	0	1

L3 cache replacement	LRU	DRRIP	SHiP	Hawkeye	SRRIP
L3C = LRU	1	0	0	0	0
L3C = DRRIP	0	1	0	0	0
L3C = SHiP	0	0	1	0	0
L3C = Hawkeye	0	0	0	1	0
L3C = SRRIP	0	0	0	0	1

B. Building IPC Prediction Models

In the next we describe the definition of IPC prediction models based on the above dataset. We will not detail the implementation of the Oracle mentioned in Section III. We use the IPC dataset to create a lookup table as the Oracle. This table enables us to retrieve the actual IPC scores from the dataset for any feature vector, i.e., benchmark name and mechanism configurations. The lookup table is encapsulated in a suitable Python wrapper, providing a compatible interface for XAI methods straightforward application to Oracle.

Baseline fully-connected neural network. We adopt the usual empirical approach, which involves extensive exploration of various model configurations to define a suitable NN. As our dataset is in simple tabular form, feed-forward NN is acceptable. In particular, we consider a fully-connected NN, which is gradually modified w.r.t. its layer count and layer size. When the performance of the NN seems adequate, we proceed to further parameter tuning. The architecture template of our NNs consists of one input, five hidden, and one output layers, as shown in Fig. 2. The input layer receives the microarchitecture features after one-hot encoding (benchmark identifiers are removed). Its number of neurons is therefore 15. An overview of the remaining layers is shown in Fig. 2.

For training each specific NN, we deliberately use around 99% of the dataset corresponding to each benchmark, i.e., 480 computer architecture configurations. The reason is that we want to mimic the corresponding data subset as closely as possible. However, we use dropout within each NN to prevent specific neurons from relying too heavily on particular features in input data. Thanks to this mechanism, certain neurons are randomly "dropped out" or temporarily removed during the training process from the NN with a certain probability.

Hyperparameter selection. Hyperparameters are crucial for optimizing NN training. Here, the main hyperparameters are the optimizer, the learning rate, the loss function, the number of training epochs, the activation functions, and the dropout probability. We used *Adam* (Adaptive Moment Estimation) optimizer [42], which combines the advantages of *stochastic gradient descent* and *root mean square propagation* to efficiently

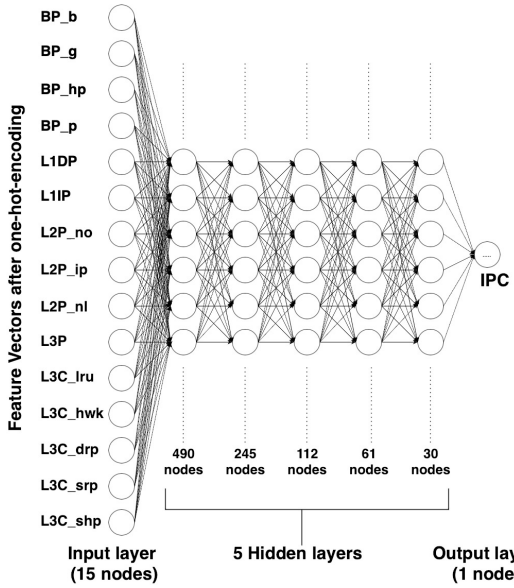


Fig. 2. The NN architecture deployed to predict IPC scores using 15 features based on 480 computer architecture variations on 49 standardized benchmarks. For the sake of concision, we adopt the following encoding: `b` = bimodal, `hp` = hashed perceptron, `p` = perceptron, `no` = no prefetcher, `nl` = next line prefetcher, `ip` = ip-stride prefetcher, `lru` = LRU, `hwk` = Hawkeye, `srp` = SRRIP, `drp` = DRRIP, and `shp` = SHiP.

update model parameters. *Mean squared error* (MSE) is used as loss function. It is commonly used in regression tasks. After evaluating different learning rates, we found 0.005 as the most appropriate value. We trained the different NNs for 350 epochs, with a batch size of 10. To avoid overfitting during training, we use the *early stopping* regularization technique with a *patience* parameter equal to 15. This technique halts the training process when the performance on the validation data degrades. The *patience* specifies the number of epochs to wait before stopping the training once the validation performance does not improve. The dropout probabilities leading to the highest NN learning performance are 0.4 and 0.3. We use the *ReLU* activation function within all layers, except for the output layer, which is configured with a *linear* function. ReLU has often proved very effective in NN training. The linear activation function is usually adopted in the output layer of a NN for certain regression tasks.

Based on the aforementioned hyperparameters, we constructed 49 NNs, each dedicated to different program benchmarks. While these NN models share the same global architecture, they differ in their weights from one benchmark to another. These weights are configured upon training every NN on 480 data samples for each benchmark.

Accuracy evaluation. Beyond the MSE-based evaluation of NN accuracy during the above training process, we further assess the precision of the resulting NN models. For this purpose, we consider the R2 score, which is a statistical measure used to evaluate regression model fitness. R2 score ranges from 0 to 1, with higher values indicating a better fit. Fig. 3 summarizes the overall results. There are 24 NNs with an R2 score greater than 0.99. There are only three NNs with a score lower than 0.9:

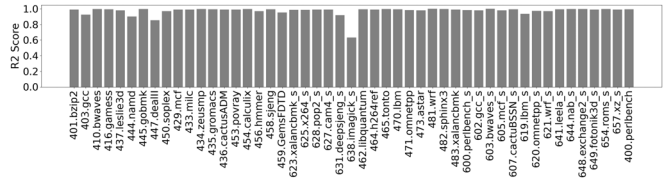


Fig. 3. R2 scores for each individual NN trained for the different benchmarks.

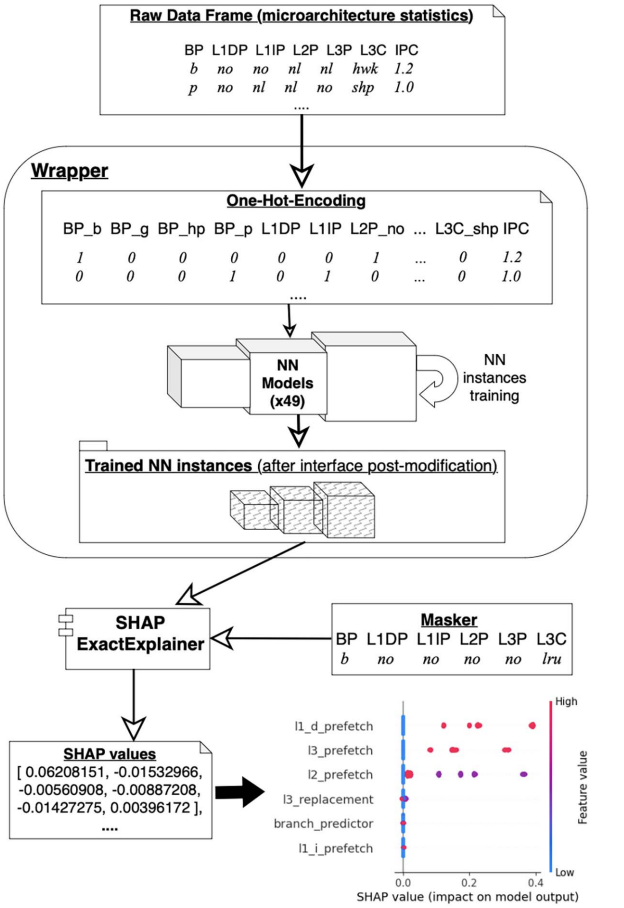


Fig. 4. NN explanation workflow in presence of one-hot encoding with SHAP.

447.dealII, 444.namd and 638.imagick_s. For the remaining 22 NNs, the R2 scores fall into the range of $0.9 - 0.99$. This shows the relevance of the NNs in general.

C. Adaptation of NN Interface to SHAP Application

One-hot encoding increases the feature space. The resulting exponential feature combinations make SHAP value computation heavier due to sparse binary vectors, i.e., with many zero values. To effectively apply the SHAP `ExactExplainer`, which exhaustively computes SHAP values, dimensionality reduction, such as feature selection or custom data preprocessing, is required. Using a specific wrapper, we implement a custom preprocessing approach, illustrated in Fig. 4. The wrapper takes raw data frames that contain feature vectors and IPC scores. The frames are transformed into one-hot encoding. Each NN associated with a benchmark is trained with the encoded feature vectors to predict IPC scores.

In the subsequent steps of the explanation workflow, the trained NNs are utilized to explain the IPC prediction. SHAP evaluates feature contribution using a *masker*, which represents a reference microarchitecture configuration. For each feature f, SHAP values are calculated based on "coalitions" of features. They indicate how significant a feature is. This is determined by checking the prediction outcome when f only is defined as well as when f and the other features are defined in an input data vector [22]. Undefined vector features must have default values. By doing so, f is computed using a relevant input data vector. Any default value can be used. NNs may, however, produce out-of-distribution samples when they have never seen such values before. The solution is to substitute undefined features with masker components. Consider a NN that predicts IPC based on branch prediction and L1-I cache prefetching but leaves the other features undefined. Masker components replace undefined mechanisms in coalition input vectors. For all cache levels (except L1-I), we select "no" for prefetching, and "LRU" for cache replacement. Calculating exact SHAP values is computationally expensive, so we limit ourselves to one masker (not a set) to avoid too many feature coalitions. In the case of feature f, the SHAP value is calculated as the average of all coalition values.

In the current study, the following masker is chosen:

BP	L1DP	L1IP	L2P	L3P	L3C
bimodal	no	no	no	no	lru

This arbitrary masker aims to express a basic microarchitecture configuration where no prefetching is present. Branch prediction and cache replacement are configured with *bimodal* and LRU respectively. It gives us insight into how features corresponding to microarchitecture mechanisms influence IPC prediction w.r.t. this reference configuration. Limiting our study to a single mask helps us strike a balance between comprehensibility and analytical efficiency. Employing additional maskers could provide further insights, albeit at the cost of increased SHAP analysis efforts.

In the final step of the workflow, we obtain SHAP values indicating the impact of each microarchitecture mechanism on IPC prediction by NNs. This is for each benchmark.

D. SHAP Model Explanation: Fidelity Between NNs and Oracle

We evaluate both Oracle and the above NN models with SHAP `ExactExplainer`[3] individually. So, we had 49 pairs of SHAP explanations computed for benchmarks in our dataset. The application of `ExactExplainer` to each NN instance takes about 4–5 minutes.

To check the fidelity of the trained NNs and the Oracle model w.r.t. SHAP explanations, we consider SHAP *summary* plots

[3]https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/explainers/Exact.html

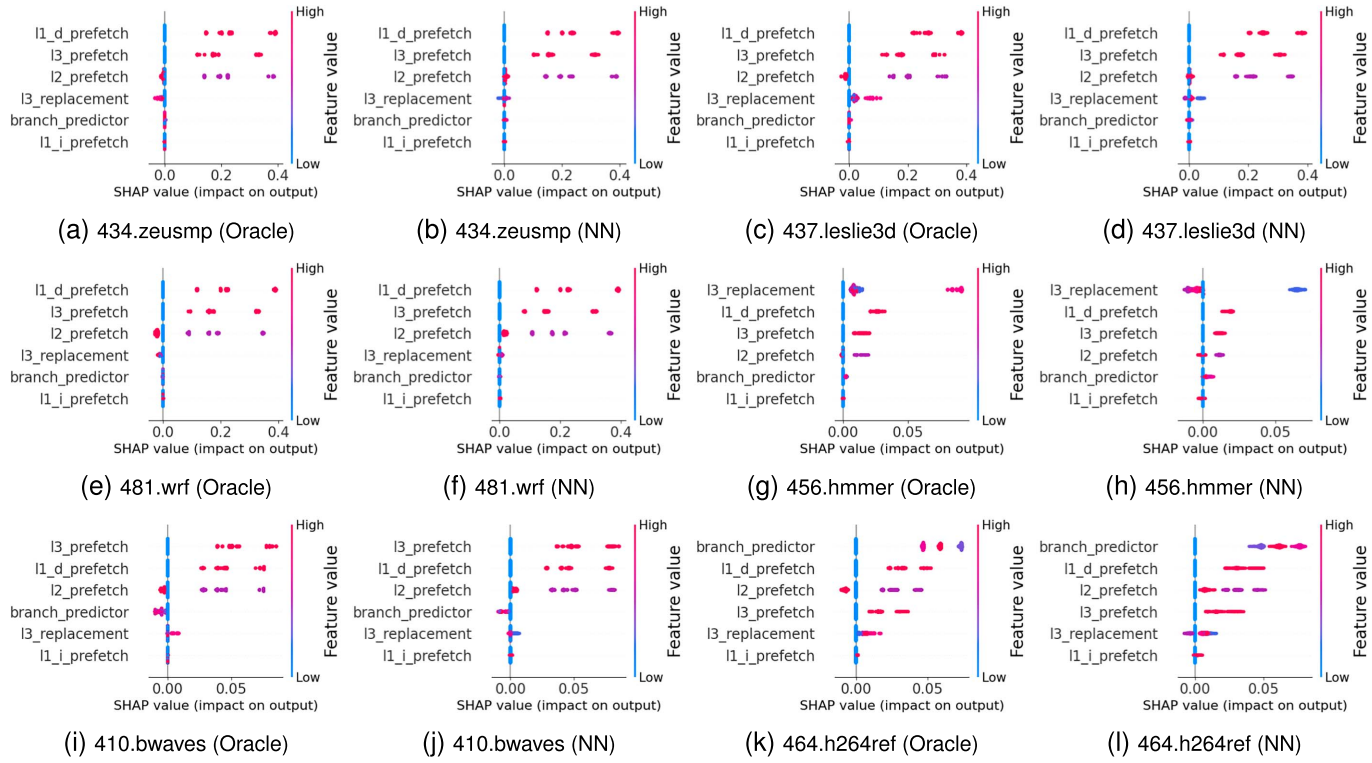


Fig. 5. Benchmarks with perfect correlation in terms of feature importance ranking for NN and Oracle. Chose 6 out of 6 pairs considered perfect.

defined by the so-called *beeswarm* plots. Such plots report the importance of each feature, as well as the associated effects on the target variables. Such a plot is a data visualization support designed to provide insights into how individual features, here microarchitecture mechanisms, contribute to model predictions. It is a type of categorical scatter plot in which the Y-axis typically represents features, while the X-axis represents SHAP values. Each dot on the plot represents a data instance. Its position on the X-axis indicates the SHAP value magnitude for a specific feature in that instance. Significant features have high SHAP values, i.e., those with high impact on the NN's outputs.

Using summary plots, we compare SHAP explanations for NNs against Oracle. Note that among the 49 NNs, 13 models produce SHAP explanations that loosely correlate with Oracle. In these models, NNs often order feature importance differently from Oracle. The majority of concerned benchmarks have lower R2 scores, e.g., 638.imagick_s. The remaining 36 NNs exhibit a reasonable correlation with Oracle's SHAP explanations. In Fig. 5, we present the beeswarm plots of six distinct benchmarks. For each benchmark, the Oracle result is displayed just before that of the corresponding NN. Each plot shows the order of importance for the six features, arranged from top to bottom, in decreasing importance order. This is based on their mean absolute SHAP values. It is important to note that features with global impact get higher priority, but rare data instances with high impact may get overlooked. The color bar on the right-hand side of each plot indicates the following code values, from 0 (i.e. `Low`) to the highest value (i.e. `High`) according to the mechanism[4]:

```
branch_predictor : bimodal=0, hashed perceptron=1,
                   gshare=2, perceptron=3;
   L1_d_prefetch : no=0, next_line=1;
   l1_i_prefetch : no_instr=0, next_line_instr=1;
     L2_prefetch : no=0, next_line=1, ip_stride=2;
     L3_prefetch : no=0, next_line=1;
  L3_replacement : lru=0,  drrip=1,  ship=2,
                   hawkeye=3, srrip=4
```

Though the above feature encoding is discrete, SHAP beeswarm plots adopt a continuous color bar. This limitation makes the analysis of the plots a bit tedious.

Fig. 5 shows consistent SHAP beeswarm plots for Oracle and NNs. However, SHAP values of some mechanisms may vary between plot variants. This particularly concerns cache replacement and branch prediction in 437.leslie3d, 456.hmmer, 410.bwaves and 464.h264ref benchmarks. For instance, DRRIP appears to be the most notable policy, while Hawkeye appears most influential with Oracle. As for branch prediction, a permutation is observed between gshare and hashed perceptron following their SHAP values in the plots of 464.h264ref. These differences in SHAP values from Oracle and NNs can have various reasons. On the one hand, NNs rely on approximations to make predictions. This may not accurately reflect the complex underlying relationships of the training dataset. On the other

[4]The cache replacement policy in L1-D, L1-I, and L2 is fixed to LRU by default. As a result, SHAP plots do not display these mechanisms.

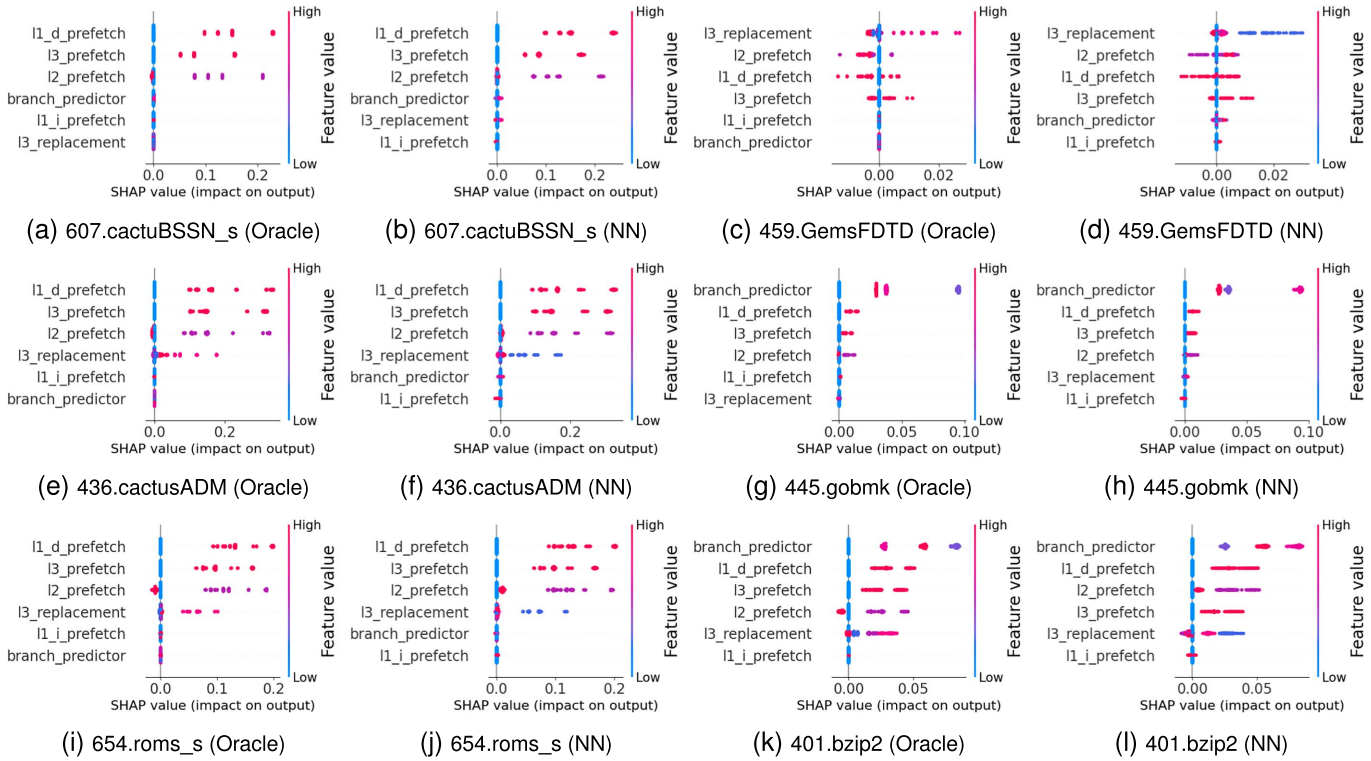


Fig. 6. Benchmarks with almost perfect correlation in terms of feature importance ranking for NN and Oracle. A maximum of one switch between features. Chose 6 out of 30 pairs considered almost perfect.

hand, it is possible that the NNs simply learned different feature interactions compared with Oracle, leading to distinct SHAP values for the same features.

Fig. 6 shows the beeswarm plots for six other benchmarks, for which the importance order of the features does not match perfectly for the Oracle and NN models as in Fig. 5. However, only one feature permutation occurs at most between the beeswarm plots variants of each benchmark. These explanations are acceptable, since they mostly yield similar feature-importance trends for Oracle and NNs. In total, 30 of the 49 benchmarks follow such trends. Similarly to Fig. 5, the SHAP values of cache replacement and branch prediction features sometimes vary between the Oracle and NN beeswarm plots.

E. SHAP Model Explanation: Design Impact Analysis

The motivation of this section is to uncover how the previous SHAP explanations correlate with common knowledge on how considered microarchitecture mechanisms affect system performance. To ensure meaningful insights, we only consider acceptable NN explanations, i.e., those of Figs. 5 and 6. In the next sections, it is critical to remember that our analysis considers a *reference microarchitecture configuration* used as *masker* in SHAP value calculation.

1) Broad Impact of Mechanisms W.R.T. IPC: From the previous NN explanations, we identify three benchmark clusters summarized in Table V. Each cluster is characterized by its most significant subset of features according to SHAP. These features are critical for improving performance. Within a cluster, benchmarks could have different feature importance orders. The three clusters and their feature subsets are:

- Cluster 1: { L1DP, L3P, L2P }
- Cluster 2: { BP, L1DP, L3P, L2P }
- Cluster 3: { L3C, L2P, L1DP, L3P }

Table V presents each cluster, the benchmarks[5] that are covered, and the favorable policies for the most influential microarchitecture mechanisms. Here, "N/A" indicates the feature does not appear in the feature set of a cluster according to the beeswarm plot of a benchmark.

Cluster 1: predominance of prefetching. This cluster points out the high impact of cache prefetching over the other microarchitectural mechanisms. This specifically concerns L1-D, L2, and L3 caches. Typical benchmarks for which this is observed in the NN model explanations include 434.zeusmp, 437.leslie3d, 481.wrf, and 410.bwaves. Their corresponding beeswarm plots, shown in Fig. 5, often place L1-D prefetching as the most influential feature, followed by L3 and L2 prefetching respectively. They also globally indicate that *next line* for L1-D, L2, and L3 caches better drives IPC scores. The same observation holds for 607.cactuBSSN_s, 436.cactusADM, and 654.roms_s (see Fig. 6). According to the dataset, the respective IPC scores of 434.zeusmp, 437.leslie3d, 481.wrf, 436.cactusADM, 654.roms_s, 607.cactuBSSN_s and 410.bwaves grow

[5]All benchmarks with a reasonable explanation correlation between Oracle and NNs were analyzed, but not all of them are included in Table V. We only report benchmarks that fall into the identified three clusters.

TABLE V
SUMMARY OF MOST INFLUENTIAL MECHANISMS PER BENCHMARK

Cluster 1: { L1DP, L3P, L2P }			
Benchmarks	L1DP	L3P	L2P
410.bwaves, 434.zeusmp, 435.gromacs, 436.cactusADM, 437.leslie3d, 473.astar, 462.libquantum, 481.wrf, 607.cactuBSSN_s, 628.pop2_s, 641.leela_s, 644.nab_s, 654.roms_s	nl	nl	nl

Cluster 2: { BP, L1DP, L3P, L2P }				
Benchmarks	BP	L1DP	L3P	L2P
403.gcc	gs	N/A	nl	nl
453.povray	gs	nl	N/A	N/A
458.sjeng	gs	N/A	nl	ip
465.tonto	gs / p	nl	nl	nl
623.xalancbmk_s	gs	nl	N/A	ip
631.deepsjeng_s	gs	N/A	N/A	ip
401.bzip2, 444.namd, 445.gobmk, 464.h264ref, 471.omnetpp, 600.perlbench_s, 620.omnetpp_s, 625.x264_s, 657.xz_s	gs	nl	nl	nl
447.dealII, 602.gcc_s, 648.exchange2_s	gs	nl	N/A	nl

Cluster 3: { L3C, L2P, L1DP, L3P }				
Benchmarks	L3C	L2P	L1DP	L3P
456.hmmer, 459.GemsFDTD	drp	nl	nl	nl
470.lbm	drp	ip	nl	nl
605.mcf_s	hwk / srp	ip	N/A	nl

by 26%, 52%, 21%, 32%, 22%, 15% and 28% in presence of *next line* in L1-D, L2 and L3 compared with the reference microarchitecture, i.e., the masker, which assumes no prefetching. Although the above observation is interesting, it is worthwhile to note that *next line* may be more detrimental to bandwidth than IP stride, e.g., in multicore designs. In general, the former consumes more bandwidth because it immediately fetches adjacent memory lines regardless of access patterns. This may result in the retrieval of more data than is necessary in non-sequential patterns. Prefetching with IP stride may be more selective, resulting in better bandwidth utilization when access patterns follow consistent strides.

Beyond L1-D, L2, and L3 prefetching, L3 replacement policy (L3C) emerges as another important mechanism among Cluster 1 benchmarks. For instance, this is observed in Figs. 5 and 6 for 437.leslie3d, 434.zeusmp, 436.cactusADM and 654.roms_s. Based on Oracle and NN explanations, Hawkeye and DRRIP are the two most noteworthy policies, respectively.

Cluster 2: predominance of branch prediction and prefetching. This cluster underlines the prime importance of branch prediction, in combination with L1-D, L2, and L3 prefetching. Concerned benchmarks include 464.h264ref (see Fig. 5), 445.gobmk and 401.bzip2 (see Fig. 6). The most impactful branch predictor is gshare according to NNs (except for 465.tonto for which perceptron is also relevant), while it is hashed perceptron for Oracle. These observations point out the limited precision of our NNs compared with Oracle. Indeed, the hashed perceptron branch predictor generally outperforms gshare. As for prefetching, *next line* is often the most effective option in L1-D, L2 and L3 caches w.r.t. IPC optimization.

Considering the data generated with ChampSim, the respective IPC scores of 401.bzip2, 445.gobmk and 464.h264ref increase by 2%, 4% and 3% in presence of gshare compared with the masker configuration, which assumes bimodal. When setting L1-D, L2, and L3 cache prefetching to *next line* in addition to gshare, the IPC scores respectively grow by 7%, 6%, and 6% compared with the masker configuration. At the same time, gshare improves the successful branch prediction ratio by 1%, 2%, and 1% respectively. Note that hashed perceptron yields slightly higher IPC scores and branch prediction ratios than with gshare according to Oracle. Our NNs cannot figure this out due to their limited precision.

In half of the benchmarks, prefetching in L1-D, L2, and L3 caches is not always among the top-4 influential features. L3 cache replacement is often the mechanism that appears in place of the missing prefetching feature, e.g., in 403.gcc and 648.exchange2_s. L1-I cache prefetching is the other feature that can occur besides L3 cache replacement in the top-4 influential features, e.g., in 631.deepsjeng_s and 453.povray.

Finally, the explanations for several benchmarks show that gshare branch prediction is often impactful. The SHAP values of the remaining three features are quasi-null, meaning their effect is quasi-null, e.g., see 648.exchange2_s and 458.sjeng.

Cluster 3: predominance of cache replacement and prefetching. This cluster exhibits a distinct feature importance subset by highlighting L3 cache replacement, and L2, L1-D, and L3 prefetching. Depending on the benchmarks, the most influential replacement policies vary among DRRIP, Hawkeye, and SRRIP. The *next line* prefetching is often the most effective. Examples of benchmarks concerned by these observations are 456.hmmer (see Fig. 5) and 459.GemsFDTD (see Fig. 6). Compared with the masker configuration, DRRIP improves the IPC score of 456.hmmer by 1%, and up to 3% in the presence of *next line* in L1-D, L2 and L3 caches.

Preliminary remarks. The analysis of the beeswarm plots enables us to identify microarchitecture mechanisms with high global impact on IPC prediction by NNs. Prefetching often ranks high. The adoption of *next line* in L1-D, L2, and L3 caches often contributes to higher IPC scores. L1-I cache prefetching has no effect. Moreover, the simultaneous presence of prefetching at all cache levels is not necessarily beneficial to performance, as observed in previous research [43]. In rare data instances, however, these broad insights can hinder exceptional situations. Typically, when branch prediction and L3 cache replacement emerge as predominant features in the beeswarm plots, the presence of L1-D, L2, and L3 prefetching has only a weak relative impact on IPC score, compared with the masker microarchitecture. To shed light on such exceptional situations, we will focus on the interactions (or synergies) between the considered microarchitecture mechanisms during IPC prediction by NNs.

2) Analysis of Mechanism Interaction: Interactions between different microarchitecture mechanisms are multiform. For instance, when a cache replacement policy evicts data likely to be needed shortly, a prefetching mechanism can anticipate and proactively load the required data into the cache, minimizing cache misses. Effective coordination between prefetching and branch prediction can occur when prefetching targets instructions and data along the predicted path. If the branch prediction

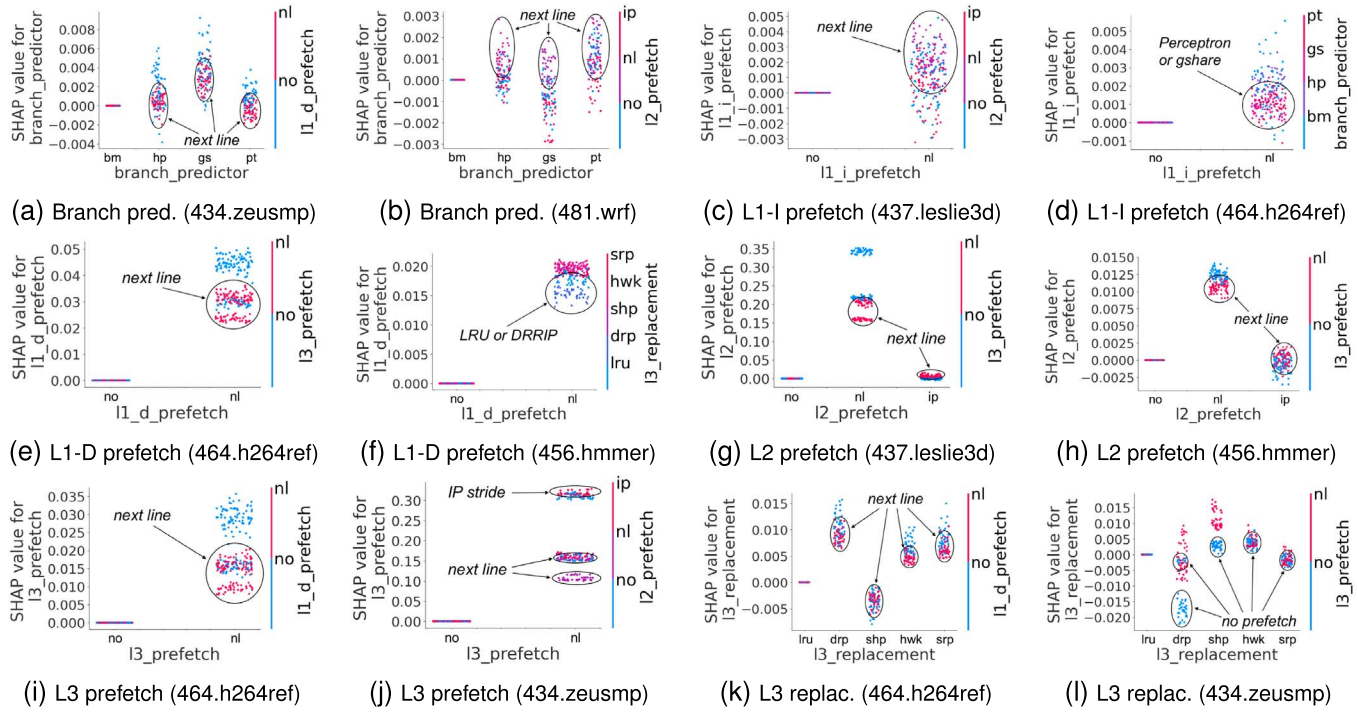


Fig. 7. SHAP dependency plots illustrating pairwise interaction between different microarchitecture mechanisms.

is accurate, the prefetched data is likely to be used, reducing cache misses and pipeline stalls. While these interactions between different mechanisms are beneficial to performance, they can also have a detrimental impact. For example, if prefetching brings in data not needed, it can fill the cache with unnecessary information, leading to cache pollution. This can result in cache thrashing, where valuable data is constantly evicted due to non-useful prefetched data. Another problematic interaction occurs when prefetching loads data along a predicted branch path, which ultimately results in a misprediction. In this case, the cache may contain data from the wrong execution path, leading to wasted memory bandwidth and potential cache pollution.

Addressing the aforementioned interactions requires a deep understanding of the specific workload characteristics and the microarchitecture design. This often involves fine-tuning and optimizing each mechanism based on workloads and processor usage patterns. To help designers, SHAP dependence plots reveal interactions or non-linear relationships between different mechanisms w.r.t. IPC predicted by NNs. A change in a given mechanism's value may have modify predictions according to other mechanisms' values. Observing how the predictions change as the mechanisms vary, one can assess the impact of the mechanism on the model's decision-making process. Finally, SHAP dependency plots help validate whether NN behavior aligns with domain knowledge and expectations and identify any inconsistencies or unexpected relationships.

Fig. 7 illustrates such plots[6] on selected mechanism interactions for benchmarks where the NN and Oracle models are highly correlated. Mechanisms are represented along the horizontal X-axis, while their associated SHAP values are plotted on the vertical Y-axis. Each dot on a plot denotes a data instance characterized by a given mechanism and its SHAP value for this instance. The color bar consists of a value range corresponding to the mechanism that interacts with that associated with the X-axis. Thus, SHAP dependency plots provide a means of analyzing pairwise feature interactions.

Dependencies of branch prediction. Branch prediction's impact on NN predictions generally fluctuates with prefetching across benchmarks. The SHAP value of branch prediction, i.e., its absolute impact on IPC prediction, marginally decreases when prefetching in L1-D or L2 is *next line* in benchmarks such as 437.leslie3d and 434.zeusmp [e.g., see Fig. 7(a)]. As a result, branch prediction impact increases either in the presence of IP stride prefetching in L2 or in the absence of prefetching in both L1-D and L2 for such benchmarks. This is particularly true when the branch prediction algorithm is gshare. However, other benchmarks exhibit an opposite trend, as shown in Fig. 7(b) for 481.wrf. Here, the impact of branch prediction marginally increases when L2 prefetching is *next line*. The most influential branch prediction algorithms are hashed perceptron and perceptron.

It is worth noting that while the average SHAP values of branch prediction vary up to about 0.1 within the beeswarm plots of Figs. 5(l), 6(h), and 6(l), here they only vary marginally between -0.004 and 0.008 in the dependency plots of Fig. 7(a) and 7(b). In other words, the relative impact of branch prediction on IPC prediction is very marginally sensitive to prefetching variation. Furthermore, bimodal always induces null SHAP values, meaning no impact, because it is the default algorithm in the masker configuration.

[6]For limited space reasons, we do not show the plots for all benchmarks.

Dependencies of L1-I cache prefetching. When prefetching in L2 and L3 is *next line*, the impact of prefetching in L1-I marginally increases for a subset of benchmarks, e.g., 434.zeusmp and 437.leslie3d [see Fig. 7(c)]. In contrast, it decreases for other benchmarks, e.g. 456.hmmer, when prefetching in L2 is *next line*. Meanwhile, we observe a correlation between L1-I prefetching and branch prediction. Typically, for 464.h264ref benchmark, the impact of *next line* prefetching in L1-I is marginally higher with bimodal than with the other branch prediction algorithms [see Fig. 7(d)]. For 456.hmmer, the highest SHAP value of prefetching is achieved in L1-I with gshare and perceptron.

Globally, the SHAP values of L1-I cache prefetching only vary between −0.003 and 0.006 in the dependency plots [see Fig. 7(c) and 7(d)]. This is consistent with our previous observations in Section IV-E1, where the small SHAP value range of L1-I prefetching indicates its very limited impact.

Dependencies of L1-D cache prefetching. The importance of prefetching in L1-D w.r.t. IPC prediction by NNs also fluctuates with prefetching in L2 and L3 caches. Typically, it increases when prefetching in L3 differs from *next line* for 464.h264ref [see Fig. 7(e)]. For other benchmarks, its influence varies with L3 cache replacement. In Fig. 7(f), the SHAP value of L1-D prefetching increases when the replacement policy is SRRIP, Hawkeye, or SHiP. It decreases otherwise. Globally, Fig. 7(e) and 7(f) reveal that *next line* prefetching in L1-D generally yields the highest impact.

Unlike branch prediction and L1-I prefetching, the SHAP value range of L1-D prefetching is between 0.01 and 0.06. This translates into its higher importance w.r.t. IPC prediction by NNs, particularly when configured as *next line*. Furthermore, the influence of L2 and L3 cache prefetching, and L3 cache replacement on the SHAP value of L1-D prefetching is non-negligible. Therefore, the synergy between all these mechanisms must be considered to optimize IPC.

Dependencies of L2 cache prefetching. The impact of prefetching in L2 on IPC prediction is tightly dependent on the prefetching configuration in L1-D and L3 caches. More generally, it increases when no prefetching is used in these caches [e.g., see Fig. 7(g) and 7(h)]. This is observed in several benchmarks, including 434.zeusmp, 437.leslie3d, 464.h264ref, 410.bwaves, and 456.hmmer. Prefetching in L2 achieves its lowest impact when set to IP stride while its most influential configuration is *next line*. The larger SHAP value range of L2 prefetching, between −0.0025 and 0.4, also indicates its notable importance w.r.t. IPC prediction by NNs.

Dependencies of L3 cache prefetching. The importance of prefetching in L3 depends on L1-D and L2 cache prefetching. More precisely, it increases when prefetching in L1-D and L2 caches is different from *next line*. This is observed with benchmarks such as 464.h264ref [see Fig. 7(i)], 437.leslie3d, 410.bwaves and 434.zeusmp [see Fig. 7(j)]. For the latter benchmark, the highest impact of L3 cache prefetching is obtained when L2 cache prefetching is set to IP stride. In general, it is achieved with *next line* prefetching in L3. Similarly to L2 prefetching, the SHAP value range of L3 prefetching confirms its importance in IPC prediction by NNs.

Dependencies of L3 cache replacement. Similarly to the previous mechanisms, the impact of L3 cache replacement depends on prefetching in L1-D, L2, and L3 caches. For some benchmarks like 464.h264ref, 410.bwaves and 456.hmmer, it decreases when prefetching is not *next line* in L1-D and L2, e.g., see Fig. 7(k). In these cases, the most impactful replacement policy is DRRIP. However, there is an exception with 434.zeusmp in Fig. 7(l), where *next line* prefetching in L3 increases the impact of SHiP cache replacement in L3.

As observed with branch prediction, the SHAP values of L3 cache replacement also vary marginally between −0.006 and 0.015 according to L1-D, L2, and L3 cache prefetching. This confirms that the relative impact of L3 cache prefetching on IPC prediction is very marginally sensitive to prefetching variation. This was observed earlier in Section IV-E1. The appropriate replacement policy depends on the benchmarks.

Closing remarks. SHAP dependency plots illustrate the affinity of the analyzed microarchitecture mechanisms. They help understand how prefetching at different cache levels affects IPC prediction. This configuration is quite dependent on the benchmarks. Prefetching at all cache levels at once rarely improves IPC according to NNs' explanations. Prefetching marginally affects the relative impact of evaluated branch prediction and L3 cache replacement policies on IPC prediction. Finally, L1-I prefetching has the lowest impact on IPC. This may be due in part to the nature of SPEC-CPU benchmarks.

More generally, all the observations resulting from the present case study could be further refined by deeply examining the algorithmic behavior of benchmarks: memory access patterns (regular or irregular), execution flow (memory or core-bound, SIMD. . .), etc. The gained insights could help one better understand how to optimize benchmark performance. Our methodology lays the foundation to create this opportunity for both compilers and computer architecture designers to optimize future systems implementation.

V. Discussion: Insights and Limitations

Traditional design analysis methods might overlook certain processor microarchitectures or fail to capture the full complexity of their component interactions. Analytical approaches are often used, with a focus on specific mechanisms interacting with other microarchitecture mechanisms [9], [32]. In the present work, we combined ML techniques, i.e. multi-layer NNs and SHAP, to devise a novel holistic methodology addressing several microarchitecture mechanisms at once. Unlike existing work on ML-based design of processor microarchitecture which mainly concentrates on accurate performance prediction [44], our proposal also accounts for prediction explainability. It allows one to confront predicted outcomes with computer architect expertise.

By visually exposing how branch prediction, prefetching, and cache replacement mechanisms interact and influence system performance in processor microarchitectures, it also empowers

designers to identify bottlenecks and areas for optimization. Typically, if SHAP analysis reveals that specific branch predictors an cache replacement strategies simultaneously create performance barriers, these parameters can be prioritized for optimization in subsequent design iterations. As a result of this process refinement, the most critical mechanisms can be addressed to enhance the overall efficiency of a design. Additionally, the useful detailed insights provided by SHAP under different workload conditions can facilitate the development of more adaptive and efficient microarchitecture management strategies to maintain high performance. This is key for making more innovative processors capable of handling the demands of future computing tasks across a range of applications and usage scenarios.

Furthermore, our proposal helps spot weak signals that may not be immediately obvious to architecture experts, yet could significantly impact processor performance. Such weak signals could be subtle changes in memory access patterns, occasional bottlenecks, or complex hardware component interactions.

On the crucial tradeoff between soundness, cost, and accuracy. To make our methodology sound, the training dataset must be relevant to the microarchitecture design problem of interest. We used ChampSim, which offers a reasonable compromise in simulation speed and accuracy for generating this dataset. NN precision also plays a critical role in design analysis. To ensure soundness, we defined an Oracle model for sanity checks regarding NN explanations. This enables us to consistently isolate a few pathological explanations, where Oracle and NNs diverge. Even though the NNs are not as accurate as the Oracle, the outcomes of both models on the analyzed mechanisms are generally consistent.

In the absence of Oracle, it is crucial to train the NNs to a high degree of accuracy. This avoids interpreting NNs with unreliable behaviors. Another model is Bayesian NN [45], which can estimate uncertainty for NN predictions. Higher uncertainty indicates less reliable predictions, which negatively affects their explanations. Alternatively, one might explore feature influence patterns based on ML models, and confirm them through data mining, e.g., through proximity or similarity relations. Lastly, domain expertise helps track suspicious explanations caused by unreliable NNs, e.g., anomalies that do not align with well-established understanding.

While our NNs do not cover all state-of-the-art microarchitecture mechanisms, they could be easily extended through incremental training on additional data accounting for additional algorithms. SHAP explanation precision depends on several factors, including the types of explainers, e.g., exact or approximate, and masker definition. Accurate SHAP analyses are generally computationally expensive. To mitigate this cost, we considered a single masker with SHAP `ExactExplainer` to favor high-quality explanations.

On the scalability of parameter interaction analysis. As of now, we have analyzed all possible dependency plots across the whole SPEC CPU benchmark suites and feature space. Many plots show common patterns, some of which are highlighted in the paper. Rather than our current manual approach, a methodical approach could make this analysis more scalable. Data mining could be used to identify clusters of workloads with similar patterns of influence on performance. Feature dependency could then be analyzed cluster-wise, in a modular way. Alternatively, some preliminary SHAP value analyses might help select key parameters or workload subsets to analyze later. For instance, in our case study the low impact of the L1-I cache prefetcher might make it a candidate feature to leave aside for dependency analysis. Similarly, dimensionality reduction techniques, like Principal Component Analysis [46], might reveal which parameters are most important and which workload subsets to focus on. This reduces the complexity of feature dependency analysis. Finally, an ongoing work [47] recently explores the notion of combined SHAP values to see how multiple features simultaneously affect a target metric. It could be further explored as a cost-effective alternative to SHAP dependency plot, capable of addressing more than two features at once.

Some non-intrinsic limitations of the proposed methodology. During our study, some SHAP explanations appeared difficult to confirm w.r.t. general knowledge of processor microarchitecture design. ChampSim's limited accuracy is one reason. For instance, the modeling of decoupled front-ends integrating early fetch engines [48] in modern processors is not yet complete in ChampSim. As a result, some effects of L1-I cache misses in ChampSim cannot be captured, which requires dedicated cache replacement [49] and prefetcher [50] to improve performance. Therefore, inconsistent simulation outputs from ChampSim could favor SHAP explanations that may not strictly align with modern processor behaviors. Considering more accurate simulators could improve insights and validate our approach under more robust conditions in the future. Beyond ChampSim, another source of inconsistency is the precision level of our trained NNs. The statistical learning process of these NNs for IPC prediction is not accurate enough to predict precise behaviors. For instance, this may explain the difference between the NN and Oracle explanations on the impact of branch prediction and L3 cache replacement (see Section IV-E1). We believe more accurate NNs could overcome this limitation. Finally, we note that the explanations in this work rely on the specific masker used with SHAP's `Exact-Explainer`. Enhancing these results could involve the use of multiple maskers to assess the importance of the mechanisms in different microarchitecture configurations.

VI. Conclusion and Perspectives

This paper presented a pragmatic data-driven methodology applying the SHAP XAI method to uncover the influence of various microarchitecture mechanisms, as well as their complex interplay within performance predictions by NNs. It shows that SHAP explanations are relevant to this goal through a typical case study. The used dataset and Python code will be made freely available for reproducibility and reusability. Many studies have shown impressive processor performance gains using ML [12], without an in-depth analysis of the rationale behind such gains. Unlike these studies, our proposal paves the way for more informed architectural design choices. This unlocks new

frontiers in ML-based processor design optimization. However, domain expertise remains crucial to reliable microarchitectural design due to ML and XAI techniques' inherent approximations. Computer architects' know-how is essential for validating some ML-based design decisions, so synergy between ML and domain experts is key.

Future work could develop this synergy by extending our methodology to case studies involving more hardware mechanisms, masker microarchitecture configurations, and multicore designs for deeper insights. Furthermore, applying our approach to low-power designs could provide valuable insights into optimizing processor energy efficiency without compromising performance. Finally, expanding our analysis to emerging workloads, e.g., machine learning and data analytics, is indeed a worthwhile direction for future work. By incorporating a more diverse set of benchmarks, one could potentially uncover novel patterns specific to modern computational challenges.

ACKNOWLEDGMENT

We would like to thank Arthur Perais and Gilles Pokam for their helpful comments and suggestions to improve this work. Thanks to Liana Keesing for contributing to the initial software framework.

REFERENCES

[1] N. Gober et al., "The championship simulator: Architectural simulation for education and competition," 2022, *arXiv:2210.14324*.
[2] C.-C. Lee, I.-C. Chen, and T. Mudge, "The bi-mode branch predictor," in *Proc. 30th Annu. Int. Symp. Microarchit.*, 1997, pp. 4–13.
[3] S. McFarling, "Combining branch predictors," Digital Western Research Laboratory, Tech. Rep. TN-36, 1993.
[4] D. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proc. 7th Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2001, pp. 197–206.
[5] D. Tarjan and K. Skadron, "Merging path and gshare indexing in perceptron branch prediction," *ACM Trans. Archit. Code Optim.*, vol. 2, no. 3, pp. 280–300, Sep. 2005.
[6] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in *Proc. 37th Annu. Int. Symp. Comput. Arch. (ISCA)*, 2010, pp. 60–71.
[7] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely, and J. Emer, "SHiP: Signature-based Hit Predictor for high perf. caching," in *Proc. 44th Int. Symp. Micro. (MICRO)*, 2011, pp. 430–441.
[8] A. Jain and C. Lin, "Back to the future: Leveraging belady's algorithm for improved cache replacement," in *Proc. ACM/IEEE 43rd Annu. Int.Symp. Comput. Archit. (ISCA)*, 2016, pp. 78–89.
[9] D. A. Jiménez, E. Teran, and P. V. Gratz, "Last-level cache insertion and promotion policy in the presence of aggressive prefetching," *IEEE Comput. Archit. Lett.*, vol. 22, no. 1, pp. 17–20, 2023.
[10] J. Kim, E. Teran, P. V. Gratz, D. A. Jiménez, S. H. Pugsley, and C. Wilkerson, "Kill the program counter: Reconstructing program behavior in the processor cache hierarchy," *SIGARCH Comput. Archit. News*, vol. 45, no. 1, pp. 737–749, Apr. 2017.
[11] L. Von Bertalanffy, "The meaning of general system theory," in *General System Theory: Foundations, Development, Applications*, vol. 30, p. 53, 1973.
[12] S. Krishnan et al., "Archgym: An open-source gymnasium for machine learning assisted architecture design," in *Proc. 50th Annu. Int. Symp. Comput. Archit. (ISCA)*. NY, USA: ACM, 2023.
[13] E. Ïpek, S. A. McKee, R. Caruana, B. R. de Supinski, and M. Schulz, "Efficiently exploring architectural design spaces via predictive modeling," *ACM SIGOPS Op. Syst. Rev.*, vol. 40, no. 5, pp. 195–206, 2006.
[14] T.-R. Lin, Y. Li, M. Pedram, and L. Chen, "Design space exploration of memory controller placement in throughput processors with deep learning," *IEEE Comp. Arch. Lett.*, vol. 18, no. 1, pp. 51–54, 2019.
[15] D. S. Lopera, L. Servadei, G. N. Kiprit, S. Hazra, R. Wille, and W. Ecker, "A survey of graph neural networks for electronic design automation," in *Proc. ACM/IEEE 3rd Workshop Mach. Learn. CAD (MLCAD)*, 2021, pp. 1–6.
[16] G. Huang et al., "Machine learning for electronic design automation: A survey," 2021.
[17] M. Rapp et al., "MLCAD: A survey of research in machine learning for CAD keynote paper," *IEEE Trans. Comput.- Aided Des. Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3162–3181, 2022.
[18] M. R. Islam, M. U. Ahmed, S. Barua, and S. Begum, "A systematic review of explainable artificial intelligence in terms of different application domains and tasks," *Appl. Sci.*, vol. 12, no. 3, 2022.
[19] N. Binkert et al., "The gem5 simulator," *SIGARCH Comp. Arch News*, vol. 39, no. 2, pp. 1–7, 2011.
[20] J. L. Henning, "Spec cpu2006 benchmark descriptions," *ACM SIGARCH Comput. Archit. News*, vol. 34, no. 4, pp. 1–17, 2006.
[21] J. Bucek, K.-D. Lange, and J. v. Kistowski, "Spec cpu2017: Next-generation compute benchmark," in *Proc. Companion ACM/SPEC Int. Conf. Perform. Eng.*, 2018, pp. 41–42.
[22] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30., I. Guyon et al., Eds., Curran Associates, Inc., 2017.
[23] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying deep learning to the cache replacement problem," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarch. (MICRO)*. ACM, 2019, pp. 413–425.
[24] E. Teran, Z. Wang, and D. A. Jiménez, "Perceptron learning for reuse prediction," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*. Piscataway, NJ, USA: IEEE Press, 2016.
[25] S. Sethumurugan, J. Yin, and J. Sartori, "Designing a cost-effective cache replacement policy using machine learning," in *Proc. IEEE Int. Symp. High-Perf. Comp. Arch. (HPCA)*, 2021, pp. 291–303.
[26] L. Peled, S. Mannor, U. Weiser, and Y. Etsion, "Semantic locality and context-based prefetching using reinforcement learning," *SIGARCH Comput. Archit. News*, vol. 43, no. 3S, p. 285–297, Jun. 2015.
[27] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, "Perceptron-based prefetch filtering," in *Proc. 46th Int. Symp. Comp. Arch. (ISCA)*, 2019, pp. 1–13.
[28] A. Zouzias, K. Kalaitzidis, and B. Grot, "Branch prediction as a reinforcement learning problem: Why, how and case studies," *CoRR*, vol. abs/2106.13429, 2021.
[29] S. Zangeneh, S. Pruett, S. Lym, and Y. N. Patt, "Branchnet: A convolutional neural network to predict hard-to-predict branches," in *Proc. 53rd IEEE/ACM Int. Symp. Microarch. (MICRO'20)*, 2020, pp. 118–130.
[30] L. Backes and D. A. Jiménez, "The impact of cache inclusion policies on cache management techniques," in *Proc. Int. Symp. Mem. Sys. (MEMSYS)*, 2019, pp. 428–438.
[31] D. Yadav and C. Paikara, "Arsenal of hardware prefetchers," *CoRR*, vol. abs/1911.10349, 2019.
[32] H.-J. Yang, J. Fang, M. Cai, and Z. Cai, "A prefetch-adaptive intelligent cache replacement policy based on machine learning," *J. Comput. Sci. Technol.*, vol. 38, no. 2, pp. 391–404, Mar. 2023.
[33] A. Seznec, "Tage-SC-l branch predictors again," in *Proc. 5th JILP Workshop Comput. Archit. Competitions (JWAC): Championship Branch Prediction (CBP)*, 2016.
[34] W. Saeed and C. Omlin, "Explainable AI (XAI): A systematic meta-survey of current challenges and future opportunities," *Knowl.-Based Syst.*, vol. 263, 2023, Art. no. 110273.
[35] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?" Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.
[36] Z. Pan and P. Mishra, "Hardware acceleration of explainable machine learning," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2022, pp. 1127–1130.
[37] T. Speith, J. Speith, S. Becker, Y. Zou, A. Biega, and C. Paar, "Expanding explainability: From explainable artificial intelligence to explainable hardware," 2023, *arXiv:2302.14661*.
[38] S. Wang, H. Yan, K. E. Isaacs, and Y. Sun, "Visual exploratory analysis for designing large-scale network-on-chip architectures: A domain expert-led design study," *IEEE Trans. Vis. Comput. Graph.*, vol. 30, no. 04, pp. 1970–1983, Apr. 2024.
[39] Y. Sun, Y. Zhang, A. Mosallaei, M. D. Shah, C. Dunne, and D. Kaeli, "Daisen: A framework for visualizing detailed gpu execution," in *Comput. Graph. Forum*, vol. 40, no. 3. pp. 239–250, 2021.

[40] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, "Berti: An accurate local-delta data prefetcher," in *Proc. 55th IEEE/ACM Int. Symp. Micro. (MICRO)*, 2022, pp. 975–991.
[41] D. A. Jimenez, "Spec CPU 2017 execution traces," 2022. [Online]. Available: https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/
[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[43] R. D.-c. Ju, A. R. Lebeck, and C. Wilkerson, "Locality vs. criticality," *ACM SIGARCH Comp. Arch. News*, vol. 29, no. 2, pp. 132–143, 2001.
[44] M. Cengiz, M. Forshaw, A. Atapour-Abarghouei, and A. S. McGough, "Predicting the performance of a computing system with deep networks," in *Proc. ACM/SPEC Int'l Conf. Perf. Eng. (ICPE)*, 2023, pp. 91–98.
[45] E. Goan and C. Fookes, *Bayesian Neural Networks: An Introduction and Survey*. Cham: Springer International Publishing, 2020, pp. 45–87. [Online]. Available: https://doi.org/10.1007/978-3-030-42553-1_3
[46] M. Greenacre, P. J. Groenen, T. Hastie, A. I. d'Enza, A. Markos, and E. Tuzhilina, "Principal component analysis," *Nat. Rev. Methods Primers*, vol. 2, no. 1, p. 100, 2022.
[47] A. Gamatié and Y. Wang, "Explainable AI for embedded systems design: A case study of static redundant NVM memory write prediction," *CoRR*, vol. abs/2403.04337, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2403.04337
[48] Y. Ishii, J. Lee, K. Nathella, and D. Sunwoo, "Rebasing instruction prefetching: An industry perspective," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 147–150, 2020.
[49] N. P. Nagendra et al., "Emissary: Enhanced miss awareness replacement policy for l2 instruction caching," in *Proc. 50th Annu. Int. Symp. Comp. Arch. (ISCA)*, 2023.
[50] G. Reinman, B. Calder, and T. Austin, "Fetch directed instruction prefetching," in *Proc. 32nd Annu. ACM/IEEE Int. Symp. Microarchit. (MICRO)*, 1999, pp. 16–27.

Abdoulaye Gamatié is currently a Senior Researcher with CNRS, affiliated with LIRMM, a joint Laboratory of University of Montpellier and CNRS, France. His research interest includes energy-efficient system design for the embedded and high-performance computing domains. He has co-authored more than 100 articles in peer-reviewed journals and conferences. He has been involved in several collaborative projects with academia and industry. He has served on the editorial boards of scientific journals like IEEE TCAD and ACM TECS.

Yuyang Wang received the M.S. degree in AI and data science from the University of Montpellier, France, in 2023. During his internship with LIRMM Laboratory, Montpellier, France he helped to deploy XAI to assist users in ML-based system design. His research interest includes AI.

Diego Valdez Duran received the B.E. degree in computer science, with a focus on AI, from Stanford University, CA, USA, in 2023, where he is currently working toward the M.S. degree in computer science. His research interest includes ML.